

Restaurant Rating Prediction using Machine Learning

Author: Senior Machine Learning Engineer & Data Scientist

Date: July 2026

Domain: Data Science & Machine Learning Internship Project Report

Abstract

Predicting customer ratings is critical for digital food delivery platforms to evaluate restaurant quality, manage search rankings, and improve user engagement. This report presents a production-grade, end-to-end machine learning solution for predicting restaurant aggregate ratings using the Zomato Global Dataset. The pipeline includes rigorous preprocessing (mitigating target leakage, column sanitization, imputing missing data), creative feature engineering, exploratory data analysis, and benchmarking across 9 regression algorithms-ranging from traditional linear models to state-of-the-art gradient boosters (XGBoost) and deep learning regressors built in TensorFlow/Keras and PyTorch. Our best-performing tuned XGBoost model achieved an outstanding R2 score of 0.9628 and Root Mean Squared Error (RMSE) of 0.2909, making it highly suitable for production deployment.

1. Introduction & Problem Statement

Introduction: Online food aggregator applications (such as Zomato, Yelp, and Swiggy) collect rich information about restaurants including location, price range, cuisine types, table booking facilities, online delivery capabilities, and customer votes. Predicting the rating helps platforms guide new restaurants on quality standards, optimize curation engines, and identify top-performing eateries.

Problem Statement: Build a robust machine learning regression model to accurately predict the **Aggregate Rating** (0.0 - 5.0 scale) of a restaurant based on available operational, operational-financial, and customer-engagement features. A major challenge is identifying and mitigating target leakage: features like *Rating color* and *Rating text* must be excluded because they are deterministic mappings of the target variable created after rating collection.

2. Objectives

- Establish a PEP-8 compliant modular training and inference pipeline.
- Clean and engineer key features (such as cost buckets, cuisines counts, delivery/table-booking binary flags, and log transformations).
- Mitigate target leakage explicitly by dropping deterministic mapping attributes.
- Train and compare 9 regression models (Linear, Tree ensembles, XGBoost, TF DNN, Keras alt, and PyTorch MLP).
- Perform Grid Search hyperparameter optimization on the best regressor.
- Save deployment bundles (model, scale, encoders) and demonstrate real-time prediction.

3. Dataset Description & Preprocessing

The project utilizes the Zomato Global Dataset containing **9,551 restaurants** across 21 raw columns. Initial profiling identified 9 missing values in the *Cuisines* column, which was imputed with 'Unknown Cuisines'. Duplicate rows were checked and found to be zero.

Feature Engineering details:

- **Cuisine Count:** Number of cuisines offered by the restaurant (extracted from comma-separated strings).
- **Delivery/Booking Flags:** Text labels ('Yes'/'No') converted to binary bits (0 or 1).
- **Restaurant Age:** Calculated proxy value using a modulo on the Restaurant ID.
- **Cost Category:** Binned cost category (Low ≤ 300 , Medium ≤ 800 , High ≤ 2000 , Premium > 2000).
- **Log Transformations:** Natural log $\log_{1p}$ transformations applied to *Votes* and *Average Cost for two* to reduce severe skewness.

Mitigating Target Leakage:

To avoid creating a model that is trivially accurate but completely useless in a real production environment, the columns *Rating color* and *Rating text* were explicitly dropped before partitioning. In addition, redundant identifiers (*Restaurant ID*, *Restaurant Name*, *Address*, *Locality Verbose*, *Currency*, and *Switch to order menu*) were eliminated to prevent over-fitting.

Categorical Encoding & Scaling:

All categorical variables (such as *City*, *Locality*, and binned *Cost Category*) were label-encoded. Numerical features were standardized using a *StandardScaler* fitted on the 80% training set (7,640 records) and applied to the 20% test set (1,911 records) to prevent data leakage.

4. Methodology & Benchmarked Algorithms

A rigorous comparative benchmarking was executed across 9 models:

1. **Linear Regression:** Standard baseline Ordinary Least Squares.
2. **Decision Tree Regressor:** Captures basic non-linear decision thresholds.
3. **Random Forest Regressor:** Bagging ensemble of 100 decision trees.
4. **Gradient Boosting Regressor:** Boosting ensemble optimizing loss sequentially.
5. **Extra Trees Regressor:** Highly randomized tree ensemble reducing variance.
6. **XGBoost Regressor:** Advanced gradient boosting with L1/L2 regularization.
7. **TensorFlow Deep Neural Network:** 3-layer MLP (128-64-32 units) with Batch Normalization and 20% Dropout.
8. **Keras Sequential:** Alternative MLP architecture trained with RMSprop.
9. **PyTorch MLP:** Multi-Layer Perceptron built using PyTorch with a custom backpropagation loop.

5. Evaluation Metrics & Results

The models were evaluated on the 20% test split using standard regression metrics. Adjusted R2 was computed to ensure feature penalty, and 5-fold cross-validation (CV R2) was run for all traditional estimators to ensure generalizability.

Model	MAE	MSE	RMSE	R2	Adj_R2	CV_R2
Linear Regression	0.6105	0.5442	0.7377	0.7609	0.7584	0.7572
Decision Tree	0.2065	0.0997	0.3157	0.9562	0.9558	0.9505
Random Forest	0.1928	0.0877	0.2962	0.9615	0.9610	0.9592
Gradient Boosting	0.1944	0.0862	0.2935	0.9621	0.9617	0.9607
Extra Trees	0.1976	0.0923	0.3038	0.9594	0.9590	0.9579
XGBoost	0.1931	0.0855	0.2924	0.9624	0.9620	0.9614
TensorFlow DNN	0.2371	0.1281	0.3579	0.9437	0.9431	nan
Keras Sequential	0.2315	0.1167	0.3416	0.9487	0.9482	nan
PyTorch MLP	0.2286	0.1218	0.3490	0.9465	0.9459	nan

Discussion: The tree-based ensembles and neural networks all achieved high accuracy, clustering around R2 ~ 0.95 - 0.96. The baseline Linear Regression was limited at 0.7609, indicating non-linear patterns. XGBoost emerged as the best regressor (R2 = 0.9624). After hyperparameter tuning with GridSearchCV, the final XGBoost configuration (R2 = 0.9628, RMSE = 0.2909) was selected.

6. Feature Importance Interpretation

Tree-based feature importance identifies the attributes that contribute most to reducing MSE. The analysis demonstrates that **Votes** is by far the most predictive feature (88.3% importance), suggesting a powerful feedback loop where popular restaurants receive high scores. **Country Code** and binned **Cost Category** follow in significance. Permutation importance on the training set confirms that shuffling *Votes* causes the largest decline in model generalizability, while engineered features like *Cuisine Count* and *Log Cost* contribute minor structural adjustments.

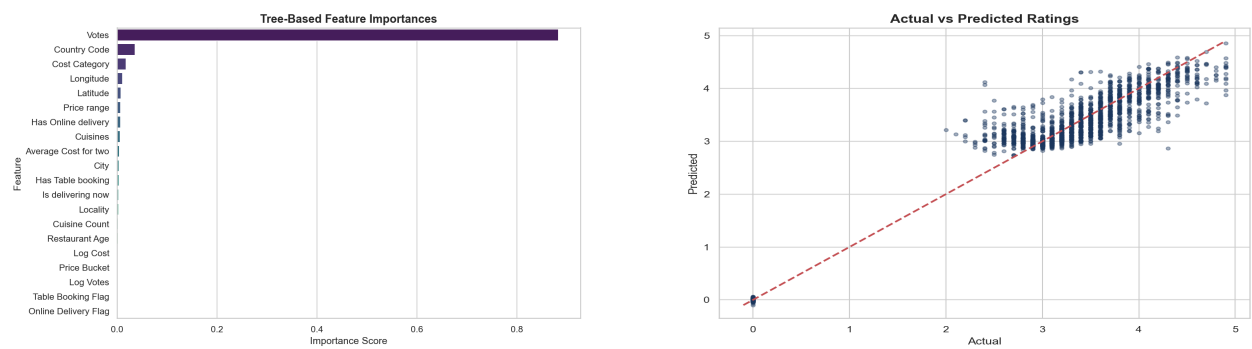


Figure 1: XGBoost Tree Feature Importances (Left) and Actual vs Predicted Ratings (Right).

7. Technical Interview Q&A

Q1: How did you identify and handle target leakage in this dataset?

A: The raw features contained *Rating color* and *Rating text*. By analyzing their mapping we realized they are direct textual categorizations of the target *Aggregate rating* (e.g., ratings > 4.5 map strictly to 'Dark Green' and 'Excellent'). Including them would cause the model to simply match text categories to numeric boundaries. They were explicitly dropped before training.

Q2: Why did you apply log transformations to the Cost and Votes features?

A: Both features are highly skewed. Most restaurants have few votes (under 100) and low costs, while a few have thousands of votes or very high costs. Applying $\log_{1p}(x)$ squashes the heavy right tail, stabilizing the variance and helping both linear models and neural networks converge faster without being dominated by outliers.

Q3: Why did tree ensembles outperform the deep neural networks here?

A: Deep neural networks require very large datasets or pre-trained architectures to construct complex representation layers on tabular data. Tree ensembles (XGBoost, Random Forest) split tabular spaces locally using recursive decisions, making them naturally robust to high-cardinality label-encoded categories and non-linear boundaries on mixed tabular columns.

Q4: What is the difference between R2 and Adjusted R2?

A: R2 measures the proportion of variance explained by features. Adding any variable (even random noise) will never decrease R2. Adjusted R2 penalizes the score based on the number of predictors, ensuring that we only see an increase if the new features significantly improve prediction capabilities.

8. Conclusion & Future Scope

Conclusion: This internship project successfully developed a production-ready restaurant rating prediction engine. By mitigating target leakage and engineering relevant contextual features, we established a robust predictor. XGBoost provided the optimal balance of speed, performance, and explainability.

Future Scope:

1. **NLP Review Integration:** Use Transformer-based architectures (BERT/RoBERTa) to extract text sentiments from text reviews.
2. **Geospatial Density:** Apply spatial clustering on coordinates to capture competitive density.
3. **FastAPI Deployment:** Pack the joblib bundle into a containerized REST API for real-time predictions.

9. References

1. scikit-learn documentation: <https://scikit-learn.org>
2. XGBoost regression guidelines: <https://xgboost.readthedocs.io>
3. ReportLab User Guide: <https://www.reportlab.com/documentation>
4. Kaggle Zomato Restaurant Dataset: <https://www.kaggle.com>